Statistical Methods in AI – Monsoon 2026

Assignment 4

MLP, IOC-NN and Clustering

Instructors: Prof. Saikiran Bulusu, Prof. Anoop M. Namboodiri

Deadline: 28th October

Marks: Q1: 100 Q2: 100 Total: 200

Rules and Submission

- **Implementation.** Use Python and NumPy with `float64`. Implement the neural networks, clustering algorithms, evaluation metrics and data splitters yourself. NumPy linear algebra, Python standard-library utilities, plotting and image I/O are allowed. PyTorch, TensorFlow, JAX, automatic differentiation, library model/clustering implementations and pretrained models are not allowed. The only exception is Q1 Section 1.4, where you may use `torch`, `torch.nn`, `torch.optim` and `torch.utils.data` to build and train networks. For Q1, `wandb` is allowed for tracking, `datasets` only for loading CIFAR-10, and `huggingface_hub` only for uploading or downloading your own saved models.
- **Random numbers.** Use NumPy's `np.random.default_rng` or Python's `random` module. For Q1, set `seed = roll_number % 100000`; for example, $2023102053\%100000 = 2053$. For run r ($r = 0$ unless specified otherwise), use:
  - `rng_split = np.random.default_rng(seed + 100*r + 0)` for splits;
  - `rng_init = np.random.default_rng(seed + 100*r + 1)` for weights;
  - `rng_batch = np.random.default_rng(seed + 100*r + 2)` for batch shuffling;
  - `rng_data = np.random.default_rng(seed + 100*r + 3)` for circles data.

  Print the Q1 seed at the start of `mlp.ipynb`. For Q2, use the seeds and initialization specified in its questions.
- **Model selection.** Use Q1's validation set and Q2's development images to choose models, features and settings. Use test data only to report final results (the one exception is the test curves of Task 1.4.2). Log all Q1 training runs to WandB.
- **Results.** Include the requested tables, figures and short answers. Plots need titles, labelled axes and a legend when comparing curves; image panels need image IDs and method/settings. Q2 ideas need not improve IoU to receive full marks if the implementation and comparisons are correct.
- **Submission.** Submit executed `mlp.ipynb` and `clustering.ipynb`, organized by question number, with their outputs visible. Submit any self-written modules they import. Both notebooks must run from a fresh kernel. Provide one `README.md` with your public WandB project link and Hugging Face links to the two selected models compared in 1.5. A separate report is not required.
- **AI-tool use.** Disclose AI-tool use in `README.md` with a shareable conversation link or export, omitting private information. Be prepared to explain every submitted line and modify your code during evaluation.

1 MLP and Input-Output Convex Neural Networks [100 marks]

1.1 Theory: Multi-Layer Perceptron [15 marks]

For an input row vector x, an MLP with L layers computes

$$h^{(0)} = x, \qquad z^{(\ell)} = h^{(\ell-1)}W^{(\ell)} + b^{(\ell)},$$

$$h^{(\ell)} = \varphi(z^{(\ell)}) \quad \text{for } \ell = 1, \ldots, L-1, \qquad o = z^{(L)}.$$

Here, $W^{(\ell)}$ has shape (number of input features, number of output neurons), $b^{(\ell)}$ is the bias vector, and φ is the activation function used in the hidden layers. The output layer does not apply an activation function.

For a classification problem with K classes, the output layer has K neurons. Its outputs $o_1, \ldots, o_K$ are called *logits*. Softmax converts these logits into class probabilities:

$$p_k = \frac{e^{o_k}}{\sum_{j=1}^{K} e^{o_j}}.$$

For an example whose correct class is $y \in \{0, \ldots, K-1\}$, the cross-entropy loss is

$$\ell = -\ln p_y.$$

For a batch of examples, the loss is the mean of the individual losses.

1.1.1 Softmax and Cross-Entropy Loss [5 marks]

(a) *(2 marks)* Starting from the definitions of softmax and cross-entropy, derive the following expression for the loss:

$$\ell = \ln\left(\sum_{k=1}^{K} e^{o_k}\right) - o_y.$$

Next, show that subtracting the same constant c from every logit leaves the softmax probabilities unchanged. Why is choosing $c = \max_k o_k$ useful when implementing softmax numerically?

(b) *(3 marks)* Derive the gradient of the cross-entropy loss with respect to each logit o_k. Show that

$$\frac{\partial \ell}{\partial o_k} = p_k - \mathbf{1}[k = y],$$

where $\mathbf{1}[k = y]$ equals 1 if $k = y$ and 0 otherwise. Show your differentiation steps.

1.1.2 Forward and Backward Pass by Hand [10 marks]

Consider an MLP with two input features, one hidden layer containing two ReLU neurons, and an output layer containing three neurons (one for each class). Thus, $L = 2$ and $K = 3$.

The ReLU activation is defined as $\varphi(t) = \max(0, t)$.

The network parameters are

$$W^{(1)} = \begin{pmatrix} 0.5 & -0.3 \\ 0.2 & 0.4 \end{pmatrix}, \qquad b^{(1)} = (0, -1),$$

$$W^{(2)} = \begin{pmatrix} 0.6 & -0.2 & 0.1 \\ -0.4 & 0.3 & 0.5 \end{pmatrix}, \qquad b^{(2)} = (0.2, 0, -0.1).$$

For the input $x = (1, 2)$, the correct class is $y = 1$ (classes are numbered $0, 1, 2$).

Let

$$\delta^{(\ell)} = \frac{\partial \ell}{\partial z^{(\ell)}}.$$

You may use the following backpropagation formulas:

$$\delta^{(2)} = p - \text{onehot}(y),$$

$$\delta^{(\ell)} = \left(\delta^{(\ell+1)}(W^{(\ell+1)})^\top\right) \odot \varphi'(z^{(\ell)}),$$

$$\frac{\partial \ell}{\partial W^{(\ell)}} = (h^{(\ell-1)})^\top \delta^{(\ell)}, \qquad \frac{\partial \ell}{\partial b^{(\ell)}} = \delta^{(\ell)}.$$

Here, $\text{onehot}(y)$ is a vector with 1 at the correct class and 0 elsewhere, and $\odot$ denotes element-wise multiplication.

(a) *(3 marks) Forward pass:* Starting with the given input and parameters, compute $z^{(1)}$, $h^{(1)}$, the output logits o, the softmax probabilities p, and the cross-entropy loss ℓ.

(b) *(5 marks) Backward pass:* Using the result from the previous section, compute $\delta^{(2)}$ and $\delta^{(1)}$. Then calculate the gradients of all four parameter vectors/matrices:

$$\frac{\partial \ell}{\partial W^{(1)}}, \quad \frac{\partial \ell}{\partial b^{(1)}}, \quad \frac{\partial \ell}{\partial W^{(2)}}, \quad \frac{\partial \ell}{\partial b^{(2)}}.$$

One of the hidden neurons receives zero gradient. Identify this neuron and explain why its gradient is zero.

(c) *(2 marks) Gradient descent:* Perform one gradient descent update with learning rate $\eta = 0.1$, using the gradients computed above to update all parameters simultaneously. Recompute the loss for the same input and label using the updated parameters. Did the loss decrease?

Show your working. Keep full precision during intermediate calculations and report numerical answers to four decimal places. You may verify your calculations using NumPy afterwards.

1.2 MLP from Scratch: Implementation and Analysis [50 marks]

1.2.1 Dataset and Preprocessing [5 marks]

CIFAR-10 has 60,000 colour photos of size 32×32 pixels in 10 classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship and truck (6,000 images per class). The task is to name the object in the picture.

(a) Loading [1 mark]: Load the dataset from Hugging Face: `from datasets import load_dataset`, then `ds = load_dataset("uoft-cs/cifar10")`. It has a `train` part (50,000 images) and a `test` part (10,000 images). Every example has an image `img` (a 32×32 colour image) and a class number `label` ($0, \ldots, 9$ in the order listed above: 0 airplane, 1 automobile, ..., 9 truck). Convert the data to NumPy arrays: join the train and test parts into one set of 60,000 images, so that `X` has shape $(60000, 32, 32, 3)$ and type `uint8` (numbers from 0 to 255), and `y` has shape $(60000,)$. (Hint: `np.array(img)` turns one image into an array; `ds.with_format("numpy")` may also help. Check the shapes.) Print the shape and type of `X` and `y` and the number of images per class. We join the two parts because you will make your own split below.

(b) Preprocessing [3 marks]:

- *Flattening.* An MLP takes a vector as input, so reshape every image to a vector of $32 \cdot 32 \cdot 3 = 3072$ numbers (`X.reshape(60000, 3072)`). The first layer of your network therefore has 3072 inputs.

- *Split.* Split the data into train, validation and test parts, *stratified*, i.e. with the same number of images of every class in each part. Write the function yourself: for every class, shuffle the 6000 indices of this class with `rng_split.permutation`; the first 3600 go to the train part, the next 1200 to the validation part and the last 1200 to the test part. You should get 36000/12000/12000*images*. Print the class counts of each part.
- *Float and scaling.* Convert the images to `float64` *before* any arithmetic (`uint8` numbers overflow), and divide by 255.
- *Standardisation.* For each of the 3072 columns, subtract its mean and divide by its standard deviation, both computed on the **training part only**; use the same numbers for validation and test. Use divisor 1 for a zero-standard-deviation column. Print the mean and the standard deviation of the first three columns for the train part and for the test part. In one sentence: why are the test numbers not exactly 0 and 1?

(c) Labels [1 mark]: keep y as integers $0, \ldots, 9$; your loss function will turn them into one-hot vectors.

1.2.2 Neural Network Implementation from Scratch [18 marks]

Build a small neural network library in NumPy. There is no fixed number of layers or widths: your code must work for any list of layers (you decide what to use in the later tasks). Use matrix operations; do not loop over examples or over single weights.

Core Components

(a) Linear Layer Class [4 marks]: Implement a `Linear` layer class that takes the input width, the output width and an activation function as arguments. It must store all data required for both forward and backward passes, including:

- Forward pass data: weights, biases (and the layer input and the values before the activation).
- Backward pass data: layer outputs, cumulative gradients.

The weights start as random numbers from a normal distribution with mean 0 and variance $2/n_{in}$ if the activation is ReLU, and variance $1/n_{in}$ otherwise (n_{in} = input width of the layer); the biases start at 0. Take the random numbers from `rng_init`.

(b) Activation Function Classes [3 marks]: Implement each of the following activation functions as a separate class. Each class must contain methods for both the forward pass (calculating the activation) and the backward pass (calculating the derivative).

- ReLU
- Tanh
- Sigmoid
- Identity (no activation)

(c) Model Class [6 marks]: Implement a `Model` class that accepts a list of `Linear` layers and a loss type.

- The model must implement `forward` and `backward` functions that execute the corresponding methods of each layer in sequence.
- Implement three losses, all as the **mean over the batch**: Mean Squared Error (MSE, the mean of (output − target)^2 over all elements); Binary Cross Entropy (BCE, for one output neuron with a Sigmoid activation: $-[\, y \ln p + (1-y) \ln(1-p) \,]$, clip p to $[10^{-12}, 1-10^{-12}]$); and Categorical Cross Entropy (CCE, the softmax cross-entropy of Task 1.1.1; it takes the logits of the last layer (Identity activation) and the integer labels, and must not overflow for large logits).

Additionally, implement the following methods in the `Model` class:

- `train(x, y)`: performs a forward pass on the input `x` to get a prediction y_{pred}. It then computes the loss between y_{pred} and the true label `y`, backpropagates the loss, and adds the resulting gradients to the cumulative gradient stored in each layer. **Do not update the parameters in this step.** It returns the loss.
- `zero_grad()`: resets the cumulative gradient in each layer to zero.
- `update(lr)`: updates all model parameters (plain gradient descent) using the cumulative gradient, and then calls `zero_grad()`. If `train` was called m times since the last `zero_grad()`, use the *average* of the m accumulated gradients (divide by m).
- `predict(x)`: performs a forward pass on the input `x` and returns the prediction y_{pred} (for the CCE loss it returns the class probabilities, i.e. the softmax of the logits).
- `save_to(path)` and `load_from(path)`: these methods should save/load all model parameters to/from a file in `.npz` format. Upload the selected MLP and IOC-NN compared in 1.5 to Hugging Face, together with their architecture and preprocessing information, and put the links in `README.md`. Loading should throw an error if the architecture of the current model (i.e., the shapes of the weight and bias arrays) does not match the shapes of the arrays in the saved file.

(d) Training Procedure [3 marks]: Implement a training loop that takes `batch_size` and `grad_accumulation_steps` (and the learning rate and the number of epochs) as parameters.

- At the start of every epoch, shuffle the training set (use `rng_batch.permutation`; create `rng_batch` once, before training) and cut it into batches.
- For each batch, call the `train()` method.
- After `grad_accumulation_steps` batches have been completed, call the `update()` method to update the parameters. (If some gradients are left at the end of an epoch, call `update()` once more.)
- Keep track of the loss at each iteration. At the end of every epoch compute the loss and accuracy on the whole training set and on the whole validation set.
- **Logging:** all training runs are logged to WandB; nothing else is needed (no separate log files, plot files or run folders). Log the hyperparameters as the run config, the training loss every 50 iterations, and the train/validation loss and accuracy after every epoch.
- **Note on Plotting:** When plotting the loss versus samples graph, each data point a model trains on is considered a sample. Ensure that the x-axis is the number of training samples seen so far (not the number of updates), so that runs with different `batch_size` and `grad_accumulation_steps` can be compared.

(e) Early Stopping [2 marks]: Implement an early stopping mechanism that uses the validation loss L_t measured at the end of epoch t: stop training when $L_t \geq 0.99\, L_{t-\text{patience}}$, i.e. when the validation loss has dropped by less than 1% over the last `patience` epochs. The parameters `patience` (default 5) and `relative_loss_threshold` (default 0.01) should be configurable. Apply the check only once $t \geq$ patience, with L_0 the validation loss before training. Replace 0.99 by $1 -$ relative_loss_threshold when the threshold is changed. Setting `patience` larger than the number of epochs switches early stopping off.

1.2.3 Sanity Check [10 marks]

Before using your code on CIFAR-10, test it on a small problem where you know what should happen.

1.2.3(a) The Concentric Circles Problem [6 marks]

(a) Create the dataset [1 mark]: Generate 2,000 points in the plane. For class 1 (inner circle) take 1000 angles θ uniformly in $[0, 2\pi)$ and the points $(0.5\cos\theta,\ 0.5\sin\theta)$. For class

0 (outer circle) do the same with radius 1. Then add Gaussian noise (mean 0, standard deviation 0.05) to both coordinates of every point. Use `rng_data` for all random numbers, and split the data 60/20/20 in the same stratified way as in Task 1.2.1 (with `rng_split`; 600/200/200 points per class). The inputs are the two coordinates; no standardisation is needed. Plot the dataset (one colour per class).

(b) Train and test the model [2 marks]: Train your model with a fixed architecture (number of hidden layers, width of hidden layers) and use each of the four activation functions (ReLU, Tanh, Sigmoid, Identity) in the hidden layers. The output layer has one neuron with a Sigmoid activation; the loss is BCE. Use learning rate 0.5, batch size 32 and 600 epochs, with early stopping switched off.

(c) Explain the results [1 mark]. In 2–3 sentences, explain any activation that fails to separate the two circles, using its decision-boundary plot. Full accuracy is not required for every activation.

(d) Report with plots [2 marks]:

- A table with train, validation and test accuracy for every (architecture, activation function) pair that you trained.
- One figure with four panels (one per activation function; use an architecture of your choice, the same for all four). Each panel is a *classification plot*: colour the (x_1, x_2) plane by the class your model predicts (use a grid of points over $[-1.5, 1.5]^2$) and draw the test points on top. Also plot the training loss curve of each of these four models.

1.2.3(b) Gradient Approximation [4 marks]

While running the above, also verify that the gradients are being computed correctly. Write a function that computes the per-weight gradient

$$\nabla_{\theta_i} L(x;\theta) \;\approx\; \frac{L(x;\theta+\epsilon e_i) - L(x;\theta-\epsilon e_i)}{2\epsilon},$$

where e_i is the vector that has a 1 at the position of parameter θ_i and 0 everywhere else (so only one parameter is changed at a time). Then verify that the obtained values converge to $\nabla_\theta L(x;\theta)$ obtained via backpropagation (for small ϵ).

Use a fixed tiny network with a smooth hidden activation and a fixed input batch; document its architecture, loss and seed. For $\epsilon \in \{10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}, 10^{-6}\}$, output a table of maximum absolute gradient error and relative L_2 error $\|g_{\text{num}} - g_{\text{bp}}\|_2 / \max(10^{-12}, \|g_{\text{num}}\|_2 + \|g_{\text{bp}}\|_2)$. Do NOT approximate gradients on the full CIFAR-10 model.

1.2.4 Analysis [12 marks]

Now use your library on CIFAR-10 (the data of Task 1.2.1): the last layer has 10 neurons with Identity activation, and the loss is CCE.

(a) Train several models [3 marks]: Train at least five different architectures ***(would be useful for Task 1.2.5 as well)***. Vary the number of hidden layers, the widths and the hidden activation function (ReLU, Tanh, Sigmoid). Use the same learning rate, batch size and number of epochs for all of them (you choose; for example learning rate between 0.01 and 0.1, batch size 64, 30 epochs, early stopping on). Log every run to WandB with a name that shows the architecture. Give a table with the architecture, the activation function and the number of parameters of each model.

(b) Report the following for every model [6 marks] (on the test set unless stated otherwise); write the metric functions yourself:

- the final loss and the final accuracy on train, validation and test;
- the confusion matrix (10×10, rows = true class, columns = predicted class), drawn as

a heat map with the numbers written in the cells;

- the **class-wise accuracy, precision, recall and F1-score** of every class, and their mean over the 10 classes. For class k, call the images of class k positive and all other images negative, and count TP, FP, TN, FN. Then

$$\text{class-wise accuracy} = \frac{\text{TP} + \text{TN}}{N}, \qquad \text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}},$$

$$\text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \qquad \text{F1} = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}.$$

Use 0 if a denominator is 0.

(c) What can you infer? [3 marks] Answer in 2–3 sentences each, using your own numbers:

- Which classes are confused with each other most often (give two examples from a confusion matrix), and which classes are the easiest? Why do you think so?
- How do depth, width and activation function change the loss and the accuracy? Compare the train and the validation numbers: do you see overfitting? Which model would you choose (decide with validation numbers) and why?
- The class-wise accuracy is high for every class, even for a weak model, but the recall of some classes is low. Why? (Hint: how many images are negatives for a class?)
- Can a class have a high precision and a low recall? What does that tell you about the model for that class? Give an example from your tables or explain why none occurs.

1.2.5 Final Challenge: Make the Model Small [5 marks]

The goal is to use as few parameters as possible while keeping the accuracy as high as possible. The number of parameters is the number of all weights and biases of all layers; for example a $3072 \to 64 \to 10$ network has $3072 \cdot 64 + 64 + 64 \cdot 10 + 10 = 197{,}322$ parameters.

(a) *(1 mark)* Compare at least three distinct parameter counts, using models from 1.2.4; additional training is optional. Output their parameter counts and validation accuracies.

(b) *(1 mark)* Plot the **number of parameters** (log scale on the x-axis) against the **test accuracy** (y-axis), with one point per model. Mark which points are which models.

(c) Choose your final model [3 marks]: among the evaluated models, select the smallest whose validation accuracy is within one percentage point of the highest validation accuracy. Ties favour higher validation accuracy, then the earlier model ID. Report its number of parameters and test accuracy, and explain your choice in 3–4 sentences (where does the curve become flat? what did you try that did not help?). Use validation accuracy to choose; report the test accuracy.

1.3 Theory: Input-Output Convex Neural Networks [10 marks]

The paper *The Curious Case of Convex Neural Networks* (Sivaprasad et al.; https://arxiv.org/abs/2006.05103; we call it "the paper") modifies a Multi-Layer Perceptron (MLP) so that each scalar output score is a convex function of the input x. Such a network is called an Input-Output Convex Neural Network (IOC-NN).

We use the same layer notation as in the MLP section. The IOC-NN used in this assignment differs from the MLP in three ways:

(a) The hidden activation is ELU instead of ReLU:

$$\text{ELU}(z) = \begin{cases} z, & z > 0, \\ e^z - 1, & z \le 0, \end{cases} \qquad \text{ELU}'(z) = \begin{cases} 1, & z > 0, \\ e^z, & z \le 0. \end{cases}$$

Both ReLU and ELU are convex and non-decreasing, but ELU can also produce negative values.

(b) The weights of every layer except the first must be non-negative. That is, $W_{ij}^{(\ell)} \geq 0$ for $\ell \geq 2$, including the output layer. The first-layer weights and all biases may have either sign.

(c) The weights in layers $2, 3, \ldots$ are initially sampled from the uniform distribution on $(0, 2/n_{in})$, where n_{in} is the number of inputs to that layer. After every gradient descent update, each negative weight $w < 0$ in these layers is replaced by e^{w-5}; non-negative weights are left unchanged. This keeps all these weights positive while allowing them to be updated during training.

The convexity guarantee concerns the network's scalar output scores before any final softmax is applied.

1.3.1 What is Different, and Why the Name? [10 marks]

(a) *(3 marks)* Complete the table below by comparing the MLP from Section 1.2 with the IOC-NN described above. Use at most 10 words per cell. For the MLP's weight initialisation, use the scheme from your Section 1.2 implementation. Then identify which properties are the same for both networks.

Property	MLP	IOC-NN
Hidden-layer activation		
First-layer weights $W^{(1)}$		
Weights $W^{(2)}, W^{(3)}, \ldots$		
Biases		
Initialisation of $W^{(2)}, W^{(3)}, \ldots$		
What happens after each gradient step?		

(b) *(7 marks)* A function f is convex if

$$f(tx + (1-t)y) \leq tf(x) + (1-t)f(y)$$

for all x, y in its domain and all $t \in [0, 1]$. In 3–4 sentences, explain why this network is called input-output convex. What are the input and output, and which quantities are kept fixed when checking convexity?

Why convexity may reduce overfitting. The paper argues that the convexity constraints act as a ***self-regulariser***: a convex function cannot bend freely, so the network can represent fewer functions and finds it harder to memorise the training set (Section 3.1 of the paper). Fig. 1 on page 2 of the paper shows this on CIFAR-10. With true labels, the test loss of AllConv starts to rise after a few epochs (overfitting), while the test loss of IOC-AllConv follows its training loss. When all the training labels are randomised, an ordinary MLP reaches 100% training accuracy but only about 10% test accuracy (chance level), whereas the IOC-MLP stays at about 10% on both, so its generalisation gap is 0. With 50% of the training labels randomised, the test accuracy of AllConv first rises and then falls as it starts to fit the wrong labels, while the test accuracy of IOC-AllConv does not drop. In the next section of this assignment, we will try to replicate this experiment with an MLP and an IOC-NN.

Keep this in mind. The paper uses larger networks and its own training setup (see Section 4 of the paper). Your results may therefore differ from those reported in the paper.

1.4 IOC-NN: Overfitting Experiment with PyTorch [15 marks]

In this section you use PyTorch (`torch.nn`) instead of your own library, and the *full* CIFAR-10 data. You build an MLP and an IOC-NN in PyTorch and repeat the label-noise experiment of the paper (Fig. 1 on page 2) to check whether the IOC-NN overfits less than the MLP.

Data. Load CIFAR-10 as in Task 1.2.1 and use the two parts as they are: the `train` part (50,000 images) for training and the `test` part (10,000 images) for testing. Flatten the images, divide by 255, and standardise every column with the mean and the standard deviation of the train part, as in Task 1.2.1. There is no validation set in this section, because nothing is chosen here: the architecture and the hyperparameters come from Task 1.2.5.

1.4.1 Building the Networks with `torch.nn` [6 marks]

Here you may use PyTorch: `torch`, `torch.nn`, `torch.optim` and `torch.utils.data`. PyTorch does the forward pass, the backward pass (automatic differentiation) and the update for you; these are the same steps you wrote yourself in Task 1.2.2.

(a) *(4 marks)* Write two functions MLP and one for the IOC-NN. Both use the depth, the widths, the loss, the learning rate, the batch size and `grad_accumulation_steps` of the model selected in 1.2.5. The MLP uses the hidden activation selected in 1.2.5, with the initialization of Task 1.2.2 (normal, variance $2/n_{in}$ for ReLU and $1/n_{in}$ otherwise, biases 0). The IOC-NN follows the three changes of Section 1.3 (ELU, weights of layers $2, 3, \ldots$ uniform in $(0, 2/n_{in})$, and after every `opt.step()` each negative entry w of these weights is replaced by e^{w-5}); its first layer starts with the same weights and biases as the MLP's. After every update, assert that the smallest weight of every layer $\ell \geq 2$ is ≥ 0. For run $r = 0$ use `torch.manual_seed(seed + 1)` before building the models, and `torch.Generator().manual_seed(seed + 2)` for the batch order of the `DataLoader`.

(b) *(2 marks)* Check your models.

(i) *(1 mark)* Print the number of parameters of each PyTorch model (the MLP's must equal the count of your NumPy MLP from Task 1.2.5). Then copy the weights of your trained NumPy MLP into the PyTorch MLP (remember the transpose) and show that the logits of both implementations for 5 test images agree (maximum absolute difference below 10^{-4}).

(ii) *(1 mark)* Check convexity numerically. Take 100 random pairs of test images (x, x') and, for each of the 10 output scores f_k (the logits, before softmax), test whether $f_k(\frac{1}{2}x + \frac{1}{2}x') \leq \frac{1}{2}f_k(x) + \frac{1}{2}f_k(x') + 10^{-4}$. Report for the MLP and for the IOC-NN how many of the 100 pairs violate the inequality for at least one k.

1.4.2 Experiment: Do Convex Networks Overfit Less? [9 marks]

Fig. 1 on page 2 of the paper trains a normal network and an IOC-network on data whose training labels are partly or completely replaced by random labels, and plots the accuracy against the epoch. You repeat this with the MLP and the IOC-NN of Task 1.4.1.

Setup.

- *Noise level* $p \in \{0, 0.5, 1\}$. For $p = 0$ the training labels are the true labels. For $p = 0.5$ half of the training images get a random label. For $p = 1$ all training images get a random label (the labels then carry no information about the images).
- *How to add the noise.* For each p, create `rng_data = np.random.default_rng(seed + 3)`. Choose $\lfloor p \cdot N \rfloor$ different training images with `rng_data.permutation` ($N = 50{,}000$) and give each of them a new label drawn uniformly from $\{0, \ldots, 9\}$ with `rng_data.integers`. Only the *training* labels change; the test labels stay true. Both models use exactly the same noisy labels.

- *Training.* Train both models for each of the three values of p, so six runs in all. Use the settings of Task 1.4.1, no early stopping (there is no validation set), and train every run for the same number of epochs $E = 100$. (If a run is still improving at the end, say so in Section 1.5.)
- *Test curves.* Evaluate the test set after every epoch to draw the curves.
- *Logging.* Log every run to WandB (same project, run names such as `mlp-p0.5` and `ioc-p0.5`, with p in the config). After every epoch log, on the whole training set and on the whole test set, the loss and the accuracy, and the time of the epoch in seconds. The training loss and accuracy are measured against the labels that were used for training (the noisy ones); the test loss and accuracy use the true labels.
- *Label-fit rate* (for $p > 0$): the accuracy of the network on the training images whose label was changed, measured against the *new, random* labels they were given. A network that ignores the noise gets about 10% (it predicts the true class, which agrees with the random label one time in ten). A network that memorises the training set goes towards 100%. For $p = 1$ it equals the training accuracy.

(a) *(2 marks)* Write the noise function. For $p = 0, 0.5, 1$ print the fraction of training labels that differ from the true labels.

(b) *(3 marks)* Run and log the six trainings. Also record the label-fit rate after every epoch for $p = 0.5$ and $p = 1$.

(c) *(4 marks)* Make three figures, each with one panel per model (MLP, IOC-NN) and the training and the test curve in each panel. Add a title, labelled axes and a legend.

(i) $p = 0$: the loss against the epoch (as in Fig. 1(a) of the paper).

(ii) $p = 1$: the accuracy against the epoch, with a dashed line at the chance level 0.1 (as in Fig. 1(b)).

(iii) $p = 0.5$: the accuracy against the epoch, with a dashed line at 0.1 (as in Fig. 1(c),(d)).

1.5 Comparison and Conclusion [10 marks]

1.5.1 Comparison [4 marks]

(a) *(1 mark)* For the true labels ($p = 0$), make a table with the MLP and the IOC-NN of Task 1.4.2 side by side: number of parameters, final training loss, test loss, train and test accuracy, the gap (train accuracy − test accuracy) and the mean F1-score (use your metric functions from Task 1.2.4 on the PyTorch predictions). Also show the confusion matrix of each model on the test set.

(b) *(1 mark)* Plot the test accuracy of both models against the epoch, with the MLP and the IOC-NN in the same panel, one panel for each noise level p.

(c) *(2 marks)* From the six runs of Task 1.4.2 make one table with a row for every (noise level, model) pair and these columns: the final training accuracy, the final test accuracy, the gap (training − test), the best test accuracy over all epochs and the epoch where it occurs, the label-fit rate at the end (for $p > 0$), the first epoch with training accuracy $\geq 90\%$ (write "not reached" if it never happens), and the mean time per epoch in seconds.

1.5.2 Justify Your Result [4 marks]

In about 12 lines, explain the result you got:

- **Clean labels.** Which of the two models of the first table is better on the test set, and how large is the difference? Is the train–test gap different? Which classes changed most in the confusion matrices and F1-scores?
- **Noisy labels.** Using the training accuracy, the gap and the label-fit rate for $p = 0.5$ and

$p = 1$, which network fits the wrong labels more? Does the test accuracy of either network rise and then fall for $p = 0.5$? Compare with the **paper's self-regularization claim** (Fig. 1(b)–(d)).

- **Cost.** How many more epochs, and how much more time, does the IOC-NN need to reach 90% training accuracy on the true labels? If the IOC-NN is still improving at epoch E on noisy labels, "has not fitted yet" is not the same as "cannot fit": what would you do to find out?
- **Explanation.** Relate the result to the constraints in Section 1.3 and the idea of a smaller set of functions. Separate observations supported by your measurements from possible explanations. You used one seed per setting, so a small difference can be due to chance.
- **Expectation.** Does your result agree with what you expected? If the IOC-NN also memorised the random labels, give one possible reason. Name one limitation of your comparison.

1.5.3 Conclusion [2 marks]

In 6–9 lines, summarise what you learned from this assignment: what did the sanity checks and the gradient check tell you about your implementation, what limits an MLP on raw CIFAR-10 pixels, what did the size challenge show, what did the label-noise experiment show about overfitting in the two networks, and what would you try next?

2 Clustering [100 marks]

The IIITH media team is building *Cut It Out*, a photo editor that turns pet photographs into stickers. To make a sticker, the editor must cut the pet out of the photo. The first prototype was written entirely by an AI agent. In the first demo, the cat disappeared and the sofa stayed. The agent still claimed: "The reconstructed photograph looks almost perfect!"

A good-looking picture does not mean a correct cut-out. In this question, every method is judged by the mask it produces. You will rebuild the editor in five steps: cut out the pet with a Gaussian mixture model (GMM) in 2.1, handle camouflaged pets with better patch features in 2.2, build editable regions with agglomerative clustering in 2.3, remove stray spots with DBSCAN in 2.4, and fix the mask with five extra labels in 2.5.

Setup

Data.

You are given 18 RGB images: 12 pet photographs (`pet_01`–`pet_12`) and 6 camouflage images (`camouflage_01`–`camouflage_06`), in which the pet is hard to separate from the background by colour. Three cleanup cases (`thin_01`–`thin_03`) are used only in 2.4.

- **Development** (choose features and settings here): the odd-numbered pets `pet_01`, `pet_03`, ..., `pet_11`; `camouflage_01`–`camouflage_03`; `thin_01`, `thin_02`.
- **Test** (report final results here): the even-numbered pets `pet_02`, `pet_04`, ..., `pet_12`; `camouflage_04`–`camouflage_06`; `thin_03`.

The files are:

- `images/`: one RGB PNG per image ID.
- `public_inputs.npz`: `<id>_anchors` (the supplied anchor labels) and `<id>_eligible` (the patches that may be queried in 2.5).
- `evaluation_masks.npz`: `<id>_truth` (reference mask, per pixel) and `<id>_patch_truth` (reference label, per patch).
- `thin_structures.npz`: `<id>_truth` and `<id>_corrupted` (reference and corrupted cleanup masks; each entry is one 4×4 block).

Patches and anchors.

Each image is 96×128 pixels. Split it into non-overlapping 4×4 patches: a 24×32 grid of 768 patches. Number the patches $0, \ldots, 767$ in row-major order, so patch (r, c) has index $32r + c$. Scale RGB values to $[0, 1]$. Every method below works on patches: it takes one feature vector per patch and returns one label per patch (1 for foreground, the pet; 0 for background). A patch label is expanded to its 4×4 pixels when scoring.

Each image has eight labelled patches called *anchors*: four background (0) and four foreground (1). In the anchor array, all other patches have the label -1. In the reference masks, -1 marks pixels or patches that are ignored when scoring.

What you may use.

- Choose features and settings on the development images only. Freeze them before running the test images.
- Model fitting may use the anchors and the labels collected in 2.5. Feature extraction and query selection must not read the reference masks.
- Use the reference masks only for scoring, for the analysis in 2.2(c), and for answering queries in 2.5.

Scoring.

Let P be the predicted foreground pixels and T the reference foreground pixels, both restricted to pixels that are not ignored. Then

$$\text{IoU} = \frac{|P \cap T|}{|P \cup T|}, \qquad \text{recall} = \frac{|P \cap T|}{|T|}.$$

IoU is 1 if P and T are both empty. Recall is 0 if T is empty. *False-positive pixels* are the pixels in P that are not in T. A group mean is the arithmetic mean of the per-image scores. Every standard deviation in this question is the sample standard deviation (divisor $n - 1$).

2.1 Cutting Out the Pet with a GMM [30 marks]

First build the baseline: fit one GMM per image and read the foreground mask from it.

(a) Implement and check EM [18 marks].
Task. Implement a GMM with full covariance matrices. It must work for any feature dimension d and any number of components K. It must support two modes:

- *Ordinary fitting:* no labels.
- *Fitting with known labels:* every component belongs to a class (0 or 1). A labelled point can be explained only by components of its own class.

The function takes the data X ($n \times d$), the initial means, the class of each component, the labels (-1 for an unlabelled point) and the number of EM updates. It returns the weights π_k, means μ_k, covariances Σ_k, the responsibilities r_{ik} and the log-likelihood trace.
Follow these rules exactly, so that your results can be checked.

1. *Initialisation.* All weights are $1/K$. Every component starts with the same covariance: the maximum-likelihood covariance of all the data (divisor n) plus $10^{-4}I$.
2. *E-step.* The responsibility r_{ik} is proportional to $\pi_k \mathcal{N}(x_i \mid \mu_k, \Sigma_k)$ and sums to 1 over k. For a point with label y_i, only components of class y_i are used; the others get $r_{ik} = 0$. Compute everything in log space (log-sum-exp).
3. *M-step.* Update π_k, μ_k and Σ_k with the usual responsibility-weighted formulas. With $N_k = \sum_i r_{ik}$, $\Sigma_k = \frac{1}{N_k} \sum_i r_{ik}(x_i - \mu_k)(x_i - \mu_k)^\top + 10^{-4}I$. Covariances are not shared. If $N_k = 0$, keep the previous mean and covariance of that component and set its weight to 0.
4. *Log-likelihood.* With c_k the class of component k,

$$\mathcal{L} = \sum_{i \text{ unlabelled}} \log \sum_k \pi_k \mathcal{N}(x_i \mid \mu_k, \Sigma_k) \; + \sum_{i \text{ labelled}} \log \sum_{k:\, c_k = y_i} \pi_k \mathcal{N}(x_i \mid \mu_k, \Sigma_k).$$

   Do not divide by the class prior.
5. *Trace and output.* One EM update is one E-step followed by one M-step. The trace contains $\mathcal{L}$ at the initial parameters, then after each update. The returned responsibilities are computed at the final parameters.

Check. Run three EM updates on

$$X = ((-1, -0.2), (-0.8, 0.3), (-0.1, -0.2), (0.2, 0.1), (0.8, -0.3), (1, 0.3)).$$

Use two components with initial means $(-0.8, 0)$ (component 0, background) and $(0.8, 0)$ (component 1, foreground). Label only the first point as background and the last point as foreground.
Report.

- **Numbers:** the final weights, means, covariances and the 6×2 responsibility matrix.
- **Figure:** the four log-likelihood values against the EM update $(0, 1, 2, 3)$.

- **Numbers:** the largest row sum of the responsibilities (it should be 1), and the number of labelled points that have non-zero responsibility on a component of the wrong class (it should be 0).

(b) Segment the images [12 marks].

Task. Fit a GMM to each of the 18 images. Describe each patch by five numbers: its mean RGB, followed by $0.25\,r/23$ and $0.25\,c/31$, where (r, c) is the patch position. We call this the *colour + position* representation.

Use the following **standard GMM setting**. It is used for all later GMM fits unless a question changes it.

- $K = 6$ components. Components $0, 1, 2$ are background (class 0) and components $3, 4, 5$ are foreground (class 1). The classes never change.
- Initial means: components $0, 1, 2$ start at the feature vectors of the three lowest-index background anchors, in increasing patch index. Components $3, 4, 5$ do the same with the foreground anchors.
- Fit with known labels, using the eight anchor labels.
- Run 30 EM updates.
- The *foreground probability* of a patch is the sum of its responsibilities on components $3, 4, 5$. Predict foreground if this is strictly greater than 0.5. Then overwrite the eight anchor patches with their supplied labels.

Report.

- **Table (9 rows):** the nine test images. Columns: image ID, IoU, recall.
- **Table (4 rows):** one row per group (development pets, test pets, development camouflage, test camouflage). Column: mean IoU.
- **Figure:** for `pet_01`, four panels: the original image, the reference mask, the predicted mask and the foreground-probability map.

2.2 Camouflage: Designing Better Patch Features [20 marks]

In camouflage images the pet has almost the same colour as the background, so mean RGB is not enough. Keep the GMM unchanged and change only the patch features.

(a) Design the patch features [12 marks].
Task. Design a new feature vector for each patch. It must satisfy:

- It is computed from the image alone (no anchors, no masks).
- It has at most 10 dimensions in total.
- It has at least two appearance features in addition to mean RGB. An appearance feature describes how the patch looks (for example colour, brightness, texture or edges). Position coordinates do not count as appearance features. Keeping mean RGB is optional.

Choose the features and their parameters using the development images only. Then standardise: for each dimension you keep, subtract the mean and divide by the standard deviation, both computed over the patches of all nine development images pooled together. Drop any dimension whose standard deviation is 0. Freeze the means, standard deviations and parameters, and apply them unchanged to the test images.
Report.

- **Table:** one row per feature. Columns: name, definition, fixed parameters.
- **Number:** the final dimension d.

(b) Test the features [5 marks].
Task. Fit the GMM with the standard setting of 2.1(b) on all 18 images, using your frozen features instead of colour + position.
Report.

- **Table (9 rows):** the nine test images. Columns: image ID, IoU with colour + position, IoU with the new features, difference (new − old).
- **Figure:** for `camouflage_01`, five panels: the original image, the reference mask, the colour + position probability map, the new-feature probability map and the new-feature mask.

(c) Explain the result [3 marks].
Task. Use the reference patch labels of `camouflage_01`. This is the one place in 2.2 where reference labels may be used for analysis. Ignore patches with label -1.
Report.

- **Table (2 rows):** foreground and background. Columns: mean patch RGB, and the mean of each of two added appearance features (standardised values).
- **Explanation:** in at most 60 words, say what your features detect that mean RGB misses. If the features did not help, say what the table suggests instead.

2.3 Editable Regions with Agglomerative Clustering [15 marks]

The team also wants users to click on a region and edit it. Check whether agglomerative clusters give useful regions, or only scattered patches.

(a) Implement agglomerative clustering [8 marks].
Task. Implement agglomerative clustering with Euclidean distance. Choose single, average or complete linkage. State your choice and use it in all later parts. Your code must:

- return the full merge table: one row per merge, in order, with the two child IDs (smaller first), the merge cost (the linkage distance) and the size of the merged cluster;
- cut the hierarchy into K clusters.

Use these conventions:

- Leaves have IDs $0, \ldots, n-1$. Merge number j (counting from 0) creates the cluster with ID

$n + j$.

- If several pairs have the same cost, merge the pair whose sorted IDs (a, b), $a < b$, come first in lexicographic order.
- After a cut, number the clusters $0, \ldots, K - 1$ in increasing order of their smallest patch index.

Check. Run it on the one-dimensional data $X = (0, 2, 5, 8)$.
Report.

- **Table (3 rows):** the merge table. Columns: child IDs, merge cost, merged size.

(b) Compare region counts [5 marks].
Task. For each of the 18 images, build the hierarchy on the patch features from 2.2. Cut it at $K = 4, 8, 16, 32$. Turn each cut into a foreground mask as follows:

1. A cluster takes the majority class of the anchors inside it. A tie means background.
2. A cluster with no anchors takes the class of the anchor nearest to the cluster centroid, in feature space. If distances tie, take the anchor with the lower patch index.
3. Finally, overwrite the eight anchor patches with their supplied labels.

Select K as the value with the highest mean IoU over the nine development images. If two values tie, take the smaller K.
Report.

- **Table (16 rows):** four groups $\times$ four values of K. Columns: group, K, mean IoU.
- **Number:** the selected K.
- **Figure:** the four region maps of `pet_01` ($K = 4, 8, 16, 32$), each cluster ID in a different colour.

(c) Check whether regions are connected [2 marks].
Task. A region is easy to edit only if it is one connected piece. For every image and every K, take the cluster IDs (before the class mapping) and count the 4-connected components of each cluster on the 24×32 patch grid. Two patches are connected if they share an edge. Add up the counts over all clusters. A perfectly connected result has a total of K.
At the selected K, find the cluster of `pet_01` with the most connected components. If two clusters tie, take the lower cluster ID.
Report.

- **Table:** add a column to the table of (b) with the group mean of the total component count.
- **Numbers:** the ID, patch count and component count of that cluster.
- **Figure:** its binary region mask.
- **Explanation:** in at most 40 words, say whether this cluster is useful as a single editing selection.

2.4 Cleaning the Mask with DBSCAN [15 marks]

Some GMM masks contain stray foreground spots. Remove them with DBSCAN, without removing thin parts of the pet.

(a) Implement DBSCAN [8 marks].
Task. Implement DBSCAN. It returns the cluster label of every point and a flag that tells whether the point is a core point. Use these rules:

- The neighbourhood of a point is every point within Euclidean distance $\leq \varepsilon$, including the point itself. A point is a *core point* if its neighbourhood has at least `min_samples` points (the point itself is counted).
- Visit the points in increasing row ID. When a cluster grows, visit neighbours in increasing row ID.
- Number the clusters $0, 1, 2, \ldots$ in the order they are found. Noise has label -1.
- A *border point* is a non-core point in the neighbourhood of a core point. It joins the first cluster that reaches it. A point first marked as noise becomes a border point if a later core point reaches it.

Check. Run $(\varepsilon, \text{min_samples}) = (0.12, 4)$ on

$$X = ((0,0),(0.1,0),(0.15,0.01),(0.15,-0.01),(0.2,0),\\ (1,0),(1.05,0.02),(1.05,-0.02),(1.1,0),(4,4)).$$

Report.

- **Table (10 rows):** one row per point. Columns: cluster label, status (core, border or noise).
- **Numbers:** the total number of clusters and the number of noise points.

(b) Choose the cleanup setting [4 marks].
Task. Apply DBSCAN to the foreground patches of a GMM mask. Use the integer grid coordinates (r, c) of each foreground patch as the point (do not normalise them). Remove the noise patches and change nothing else.
Compare six options: no cleanup, and the five settings $(\varepsilon, \text{min_samples})$ equal to $(1.01, 3)$, $(1.5, 3)$, $(1.5, 5)$, $(2.1, 6)$ and $(2.1, 9)$. Test them on eleven cases:

- the nine development image masks, i.e. the final new-feature masks from 2.2(b);
- `thin_01` and `thin_02`, where the input is the corrupted mask and the target is the reference mask. Each entry is one 4×4 block, so one block is 16 pixels.

Count false positives in pixels for all eleven cases. Select the option with the highest mean IoU. If options tie, prefer no cleanup, then the smaller ε, then the smaller `min_samples`.
Report.

- **Table (6 rows):** one row per option. Columns: mean IoU, mean recall, mean false-positive pixels, mean number of removed patches (all averaged over the eleven cases). Mark the selected option.
- **Figure:** for `thin_01`, the reference mask, the corrupted mask and the cleaned mask from the selected option.

(c) Evaluate on the test cases [3 marks].
Task. Apply the selected option, without changing it, to the nine test image masks (new-feature masks from 2.2(b)) and to `thin_03`.
Report.

- **Table (10 rows):** one row per case. Columns: IoU, recall, false-positive pixels (each before and after cleanup), and the number of removed patches.
- **Explanation:** in at most 60 words, pick one case where cleanup helped, hurt, or changed nothing. Cite its scores.

2.5 Fixing the Mask with Five Extra Labels [20 marks]

If the automatic mask is still wrong, the editor lets the user label five more patches as pet or background. Compare two ways to choose those five patches: at random, and where the GMM is most uncertain.

(a) Implement two query rules [10 marks].
Task. Start from the GMM of 2.2 (new features, standard setting), before DBSCAN cleanup. Implement two rules for choosing the next patch to query:

- *Random:* choose uniformly at random.
- *Entropy:* choose the patch whose foreground probability p has the largest binary entropy $H(p) = -p \ln p - (1-p)\ln(1-p)$. If two patches tie, take the lower patch index.

A patch can be queried only if it is eligible (`<id>_eligible`), is not an anchor, and has not been queried yet. Each run makes five queries, so it queries five different patches.
Run each rule separately, starting from the original eight anchors. Repeat for $q = 1, \ldots, 5$:

1. Choose a patch with the rule, using the current fit.
2. Read its label from `<id>_patch_truth`. Read it only after the patch is chosen.
3. Add the label to the known labels (the anchors plus all queried labels).
4. Refit the GMM on all patches, using all known labels. Use the standard setting, with the initial means taken from the *original* eight anchors (do not recompute them).

After every refit, predict the mask with the standard rule, and overwrite all known-label patches with their labels. The mask before any query is the initial mask.
For the random rule, create one `np.random.default_rng(seed)` per run. At each query, call `rng.choice` on the sorted array of the remaining eligible patch indices. For `pet_01`, use the seed 20261009.
Report.

- **Table (5 rows), one for each rule:** query number, patch index, returned label, remaining budget (the number of queries still left).
- **Figure:** for `pet_01`, the initial mask and the final mask of both rules.

(b) Compare the rules [7 marks].
Task. Run both rules on all nine test images. For the random rule use 20 seeds, $20261009 + s$ for $s = 0, \ldots, 19$. For each image and seed, create a new `np.random.default_rng(seed)` and draw as in (a).
Report.

- **Table (9 rows):** the nine test images. Columns: initial IoU, random final IoU (mean $\pm$ standard deviation over the 20 seeds), entropy final IoU.
- **Figure:** mean IoU against the number of extra labels $(0, \ldots, 5)$, for test pets and for test camouflage, one line per rule. For the random rule, first average each image's curve over the 20 seeds, then average over the images of the group.
- **Figure:** the original image of `pet_02` with its five entropy queries marked and numbered in order.

(c) Recommend the final editor [3 marks].
Task. Compare the methods of this question on the test images.
Report.

- **Table (6 rows):** one row per method. Columns: IoU on the test pets and IoU on the test camouflage images, each as mean $\pm$ standard deviation across images. The methods are:
  1. GMM, colour + position (2.1(b));
  2. GMM, new features (2.2);
  3. agglomerative clustering at the selected K (2.3);

4. the selected DBSCAN cleanup on the new-feature masks (2.4);
5. random correction after five labels (2.5(b));
6. entropy correction after five labels (2.5(b)).

   For random correction, use each image's mean IoU over the 20 seeds as its score, and then compute the group statistics.

- **Recommendation:** 3–4 lines, based on the table. Treat the fully automatic methods separately from the ones that need five extra labels. Name one failure case that remains on the test images.

Pet images and masks: Oxford-IIIT Pet dataset, CC BY-SA 4.0.